Lenguajes de Programación

Primera Prueba Presencial

INTRODUCCIÓN + PROGRAMACIÓN FUNCIONAL

Fecha: 26 de octubre de 2015

Valor de la prueba: 33%

Advertencia Importante: Observa bien la sintaxis en los ejemplos. No se considerarán soluciones que utilicen funciones auxiliares y/o no mantengan la sintaxis de la llamada y de los resultados.

EJERCICIO 1 (2,5 puntos)

Sin definir otras funciones auxiliares, definir una función en lisp llamada **lista_triplicados** que acepte una lista sin anidamientos y devuelva una lista cuyos elementos son listas que contienen tres veces el elemento que estaba en su misma posición en la lista original.

```
> (lista_triplicados '())
NIL

> (lista_triplicados '(1))
((1 1 1))

> (lista_triplicados '(1 2 3))
((1 1 1) (2 2 2) (3 3 3))

> (lista_triplicados '(1 7 9 2 3))
((1 1 1) (7 7 7) (9 9 9) (2 2 2) (3 3 3))
```

EJERCICIO 2 (2,5 puntos)

Sin definir otras funciones auxiliares, escribe una función en LISP llamada **maxmin** que acepte un número variable de argumentos numéricos y devuelva una lista de dos elementos que contenga el menor y el mayor de los argumentos.

```
> (maxmin)
(NIL NIL)

> (maxmin 2)
(2 2)

> (maxmin 2 4 8)
(2 8)

> (maxmin 2 4 8 7 3)
(2 8)

> (maxmin 2 4 8 7 3 -1)
(-1 8)
```

EJERCICIO 3 (2,5 puntos)

Sin definir otras funciones auxiliares, escribe una función en LISP llamada **empareja** que, dada una lista sin anidamientos, devuelva una nueva lista cuyos elementos sean pares que contengan el primer y último elemento, el segundo y el penúltimo, y así sucesivamente. Si la lista contiene un número impar de elementos, el último par consistirá del elemento central replicado.

```
> (empareja '())
NIL

>(empareja '(1))
((1 1))

> (empareja '(1 2 3 4 5))
((1 5) (2 4) (3 3))

> (empareja '(9 6 2 1 4))
((9 4) (6 1) (2 2))

> (empareja '(9 6 1 4))
((9 4) (6 1))

> (empareja '(A B C D))
((A D) (B C))
```

EJERCICIO 4 (2,5 puntos)

Sin definir otras funciones auxiliares, escribe una función en LISP llamada **unary_functions_compositions** que, dado un valor numérico y un número variable de funciones, devuelva el resultado de componer, en el orden en que aparecen, todas y cada una de las funciones sobre el argumento numérico.

```
> (unary_functions_composition 3)
3
> (unary_functions_composition 3 #'1+)
4
> (unary_functions_composition 3 #'1+ #'1-)
3
> (unary_functions_composition 3 #'1+ #'1- #'1-)
2
```